PiSPICE: Accelerating Post-Layout SPICE Simulation via Essential Parasitic Identification

Zhou Jin[1], Jing Li[2], Jian Xin[3], Tianjia Zhou[3], Xiao Wu[4], Dan Niu[5], and Zuochang Ye[3]
[1]College of Integrated Circuits, Zhejiang University, China, [2]SSSLab, China University of Petroleum-Beijing, China
[3]Tsinghua University, China, [4]Huada Empyrean Software Co. Ltd, China, [5]School of Automation, Southeast University, China
Email: z.jin@zju.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133011

***Abstract*—As process nodes scale to more advanced technologies, post-layout simulations for integrated circuits have become increasingly complex, involving billions to trillions of nodes. The growing design complexity and transistor integration require more accurate and efficient post-layout SPICE simulations. However, existing methods for solving large-scale post-layout circuits face significant challenges due to high computational costs. In this paper, we propose a new approach, PiSPICE, which utilizes adjoint sensitivity analysis to identify critical parasitics and eliminate non-critical ones, effectively reducing the simulation scale and improving speed. By modeling parasitics and performing sensitivity analysis on pre-layout circuits, we significantly reduce the computational burden and avoid the overhead of directly analyzing sensitivities in large-scale post-layout circuits. By retaining only the critical parasitics and applying model order reduction to minimize their impact, while eliminating non-critical parasitics, PiSPICE achieves a speedup of up to 17.27x in simulation with an error margin of less than 0.78% compared to the commercial simulator Spectre.**

***Index Terms*—Post-layout SPICE simulation, Parasitic modeling, Sensitivity analysis, Essential parasitic identification, Model order reduction**

I. Introduction

As semiconductor technologies advance towards increasingly smaller process nodes, the complexity of integrated circuits (ICs) has escalated dramatically. At these small process nodes, ICs are composed of billions to trillions of transistors, and layout-induced parasitics (e.g. resistance, capacitance) become increasingly significant. These parasitics affect the circuit's performance, particularly in terms of signal integrity, power consumption, and timing, and must be accurately modeled and included in post-layout simulations [1]–[11]. However, with the increasing transistor integration and shrinking design margins, traditional post-layout SPICE simulations that account for parasitic effects are becoming computationally prohibitive, both in terms of memory and processing time.

The computational burden is primarily due to the large-scale matrix operations involved in simulating complex circuits, which often require solving sparse, high-dimensional systems of equations, particularly when dealing with millions to billions of nodes [12], [13]. To address these challenges, various techniques have been proposed. One common approach is the domain decomposition method (DDM), which divides the circuit into smaller subproblems and constructs a bordered block diagonal matrix. This enables parallel processing and simplifies the solution [14]–[18]. However, DDM faces challenges when solving for coupling nodes, i.e., the connections between subcircuits, which create a global communication bottleneck. As the number of partitions increases, this bottleneck becomes a critical limiting factor, hindering the scalability of the method.

Another widely explored technique is model order reduction (MOR), which simplifies the system by approximating it with a lower-order model that retains the dominant behavior of the circuit [19]–[25]. However, MOR methods often fail to provide sufficient accuracy when dealing with highly complex and parasitic-rich circuits in advanced technologies. Additionally, all existing methods overlook the fact that not all nodes in a circuit are equally sensitive to parasitic parameters, meaning that parasitics only significantly affect the post-layout performance of certain nodes.

In light of these challenges, we present a new post-layout simulation acceleration framework, PiSPICE, which employs sensitivity analysis to identify and retain only the most critical parasitic elements while discarding non-critical ones. PiSPICE leverages pre-layout netlists to perform parasitic modeling and adjoint sensitivity analysis. This approach avoids the computational overhead of analyzing sensitivities directly on large-scale post-layout circuits, which can be time-consuming and resource-intensive. PiSPICE identifies the essential nodes in the pre-layout circuit that are most sensitive to parasitic effects and focuses on retaining the parasitic sub-networks around these nodes. Additionally, MOR techniques are applied to minimize the impact of retained parasitics, reducing the dimensionality of the problem and optimizing the overall simulation process.

Our contributions are summarized as follows.

- To the best of our knowledge, this is the first work to accelerate post-layout simulation by identifying the significance of parasitics on circuit performance, thereby significantly reducing the simulation cost.
- Adjoint sensitivity analysis is performed on pre-layout circuit to identify essential parasitics, avoiding the costly computations of directly applying it to large post-layout circuits.
- The scale of the post-layout circuit is markedly reduced by collapsing non-critical parasitic sub-networks, and applying an improved PRIMA algorithm to critical parasitic sub-networks.
- PiSPICE is especially effective for circuits with identical

topologies but varying parameters, such as sizing, where sensitivity analysis only needs to be performed once.

• PiSPICE achieves up to 93.77x reduction in circuit scale and 17.27x speedup over Spectre, and up to 43.62x reduction in circuit scale and 9.50x speedup over Spectre with TICER.

II. Background

A. Post-layout SPICE Simulation

Post-layout simulation refers to simulating a circuit after its layout design is complete. During this stage, layout information, including parasitic resistances and capacitances, is extracted and added to the circuit model. This provides a more accurate prediction of how the circuit will behave in real-world conditions once fabricated. It is crucial for verifying the circuit's functionality and performance, addressing factors such as timing analysis, signal integrity (including noise and crosstalk), power consumption, and potential manufacturing variability [26]. Post-layout simulation is essential for design signoff, ensuring the final design meets specifications and performs reliably before fabrication.

B. Adjoint Sensitivity Analysis

Adjoint sensitivity analysis is an efficient technique in circuit simulation for evaluating the impact of circuit parameter variations on performance metrics such as output voltage, current, or power consumption [27], [28]. By solving adjoint equations, it calculates sensitivities to multiple parameters simultaneously with far fewer simulations than traditional methods, i.e., finite difference and direct methods, making it especially effective for large-scale circuits and optimization tasks. Applicable in both the time and frequency domains, this approach combines computational efficiency with accuracy, identifying critical parameters to optimize designs and ensure reliable performance in complex systems [29]–[31].

For a circuit in the form of differential algebraic equations (DAE) [32]:

$$g(x,t,p) = \frac{d}{dt}q(x,p) + f(x,p) + b(t,p) = 0 \tag{1}$$

where x is the state vector, p is the parameter vector, t is time, q means dynamic variable, f means static variable, b means inputs to system. The adjoint sensitivity (i.e., the derivative of the objective function O with respect to the parameter p) is analyzed by:

$$\frac{dO}{dp} = \sum_{i=0}^{n}\sum_{k=i}^{n}(\frac{\mathrm{d}O}{\mathrm{d}x})_k(\frac{\partial g}{\partial x})_k^{-1}\left(\prod_{j=k-1}^{i}(\frac{1}{h_t}\frac{\partial q}{\partial x})_j(\frac{\partial g}{\partial x})_j^{-1}\right)(\frac{\partial \varphi}{\partial p})_i \tag{2}$$

In recent years, advanced methodologies have been developed to enhance computational efficiency in sensitivity analysis, enabling faster processing while maintaining high accuracy. Notable examples include fast adjoint transient sensitivity analysis for multiple time points objective functions [32], time-domain sensitivity analysis approach that significantly reduces runtime without compromising precision [33], and memory-efficient adjoint sensitivity analysis leveraging compression [34]. These innovations address scalability and complexity challenges, making sensitivity analysis more suitable for contemporary circuit design tasks.

C. Model Order Reduction

Model order reduction aims to reduce the order (size) of the system while maintaining an accurate representation of the circuit's behavior, thereby accelerating simulation time.

TICER [36] is a classical MOR algorithm based on node elimination. It efficiently reduces the size of parasitic networks, ensuring the preservation of circuit stability and passivity, and is relatively simple to implement. As a result, it has been widely adopted by commercial design tools, e.g., Calibre xRC. However, as the circuit scale increases, the model reduced by TICER can become denser, leading to higher computational overhead. Additionally, its performance may be limited in scenarios that require high precision.

PRIMA [35], another MOR algorithm, is based on projection methods. PRIMA is particularly effective for sub-networks, which are typically smaller and exhibit simpler dynamics, making it efficient for capturing their behavior. For an $n \times n$ system of equations describing a network, such as:

$$C\dot{x}(t) = -Gx(t) + Bu(t) \tag{3}$$

where G is the conductance matrix, C is the capacitance matrix, and $x(t)$ is the state vector. PRIMA approximates this system by projecting onto a reduced-order subspace with a transformation matrix R, where

$$\hat{G} = R^T G R \quad \hat{C} = R^T C R \quad \hat{B} = R^T B \tag{4}$$

This reduces the size of the system from $n \times n$ to $m \times m$, where $m < n$. The projection matrix R is typically chosen to capture the most important modes of the system, often using techniques like proper orthogonal decomposition.

D. Motivation

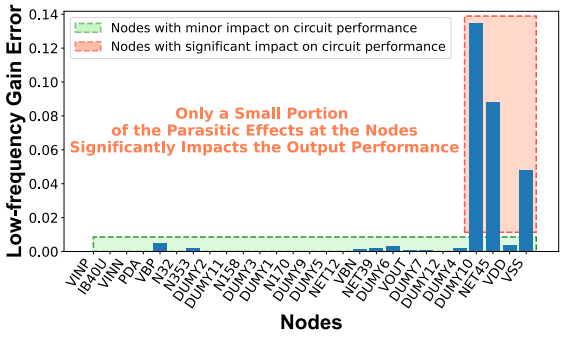

Fig. 1: Low-frequency voltage gain error of an operational amplifier by removing parasitic sub-networks for each node.

This paper is based on a key observation: in most cases, only a small portion of the parasitic effects of pre-layout nodes will significantly affect circuit performance. As illustrated in Figure 1, the majority of parasitic sub-networks have negligible effects on the circuit's output. Therefore, effectively identifying these parasitic-sensitive nodes and eliminating the less influential parasitic networks can substantially reduce the RC scale of post-layout simulations while maintaining accuracy, thereby significantly improving simulation speed.

III. Proposed Method

A. Overall Framework

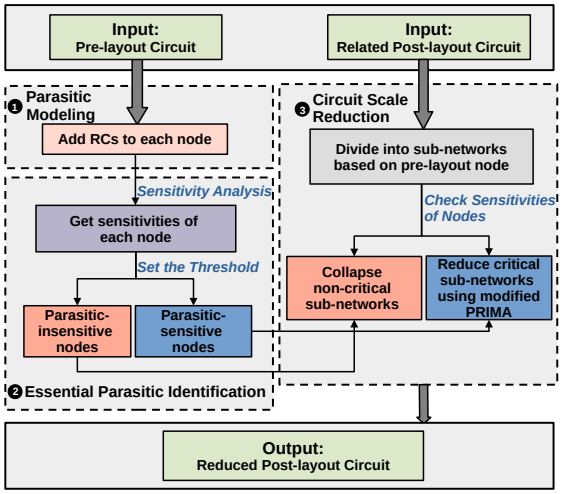

Fig. 2: The workflow of PiSPICE.

In this chapter, we introduce PiSPICE, the proposed post-layout simulation acceleration framework. PiSPICE identifies nodes that are parasitic-sensitive, those whose parasitic sub-networks significantly impact post-layout simulation performance. By retaining only the parasitic networks of these sensitive nodes, PiSPICE reduces the simulation scale and improves post-layout simulation speed. The overall framework is illustrated in Figure 2 and consists of three key components: (1) Parasitic modeling on the pre-layout circuit, (2) Perform an adjoint sensitivity analysis in this modeled circuit to identify parasitic-sensitive pre-layout nodes, (3) Reduce the RC networks according to the sensitivity results.

Performing a sensitivity analysis for every parasitic parameter in the post-layout circuit is computationally expensive. Therefore, instead of analyzing all parasitic parameters in the post-layout circuit, we focus on identifying the nodes in the pre-layout circuit that are parasitic-sensitive and target their parasitic sub-networks for further processing. To achieve this, we first model parasitic parameters using the pre-layout circuit by introducing pseudo RC elements at each circuit node to emulate the corresponding parasitic sub-networks.

Next, we perform an adjoint sensitivity analysis on this pseudo circuit to distinguish between parasitic-sensitive nodes and parasitic-insensitive nodes. Then the extracted parasitic network is partitioned on the basis of pre-layout circuit nodes. According to the results of the sensitivity analysis, insensitive networks are collapsed to significantly reduce their scale, while sensitive networks are retained. Furthermore, an improved MOR algorithm is applied to compress the size of sensitive networks, enhancing simulation efficiency. The entire process is detailed in Algorithm 1.

B. Parasitic Modeling on the Pre-layout Circuit

Parasitic networks typically consist of ground capacitance, coupling capacitance between nodes, and parasitic resistance of interconnects. To accurately model the parasitic sub-network of each circuit node, we take the following approach.

Algorithm 1 The overall framework of proposed PiSPICE.

Input: The pre-layout circuit C_{pre} with N nodes and its related post-layout circuit C_{post}
Output: Reduced post-layout circuits C'_{post}

for $j = 1$ to N **do**
  Add modeled parasitic capacitors $Caps$ and resistors $Ress$ and set there value to $p = 1e-15$
end for
Calculating the sensitivity of $Caps$ and $Ress$
for $j = 1$ to N **do**
  Calculate the mean value of the sensitivity of $Caps$ and $Ress$ to which $Node_j$ is connected
end for
Set the average sensitivity of nodes as the threshold s, and sequentially divide nodes into parasitic-sensitive nodes and parasitic-insensitive nodes
Divide C_{post} into N sub-networks
for $j = 1$ to N **do**
  Collapse the sub-network of parasitic-insensitive nodes and reduce the sub-network of parasitic-sensitive nodes using the improved PRIMA
end for

Pseudo-capacitors are placed between each node and the ground (VSS) to emulate the ground capacitance. Pseudo-capacitors are inserted between each node and all other nodes to simulate the coupling capacitance. Pseudo-resistors are added to the interconnects, with additional circuit nodes introduced to represent the parasitic resistance.

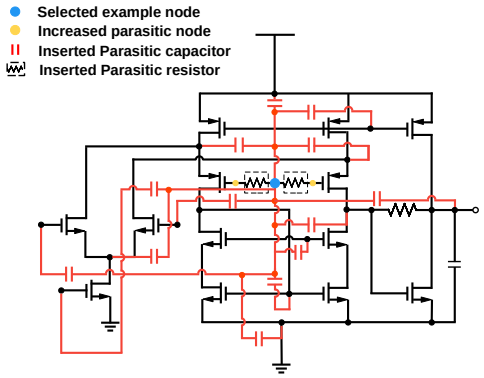

Fig. 3: An example of parasitic modeling for selected blue node. Pseudo-capacitors (red ones) are inserted between it and all other nodes to model coupling parasitic capacitance, and between it and VSS to represent ground parasitic capacitance. Resistors (in dashed box), along with corresponding new nodes (yellow ones), are added to model the parasitic resistance of interconnects.

This setup ensures a comprehensive emulation of the parasitic effects for each node's sub-network, laying the foundation for subsequent sensitivity analysis. An example is shown in Figure 3, where the blue node is demonstrated as an example to illustrate the modeling of its parasitic sub-network.

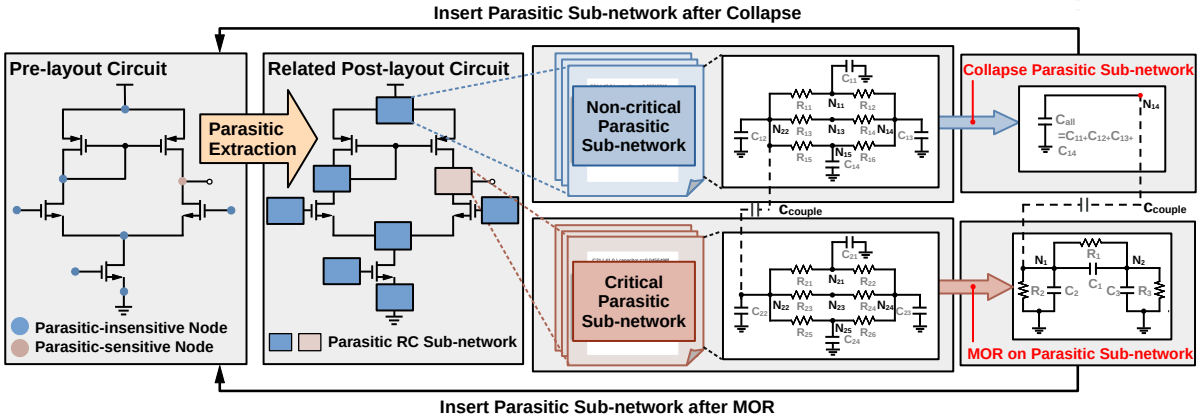

Fig. 4: After identifying parasitic-sensitive nodes, we leverage a hybrid strategy to reduce the simulation scale by handling critical and non-critical parasitic sub-networks separately.

The added parasitic resistors and capacitors are assigned identical values that are effectively close to zero. This deliberate choice ensures that the influence of these parasitic components on circuit performance is determined primarily by their topological placement, rather than their parameter values.

C. Essential Parasitic Identification with Sensitivity Analysis

After constructing the parasitic RC network for each node in pre-layout circuit, a pseudo post-layout circuit is generated. We then perform an adjoint sensitivity to this circuit to identify which nodes are highly sensitive, indicating parasitics that significantly affect the performance of post-layout simulations.

This procedure consists of two steps. First, sensitivity analysis is performed by calculating the derivative of the objective function $f(\mathbf{p})$ (the circuit's performance) with respect to the parameters $\mathbf{p}= p_1, p_2, ..., p_n$ (each parasitic component). This allows us to identify which parasitic components p_i have a significant impact on the circuit's performance, i.e., which components yield large values of $\frac{\partial f}{\partial p_i}$. However, our primary focus is on identifying circuit nodes that are parasitic-sensitive, as this information can assist in subsequent partitioning and reduction of parasitic sub-networks based on the nodes. Therefore, in the second step we calculate the sensitivity S_{node} of each node. It is defined as the average of the total sensitivity of parasitic components p_i connected to the node v_j:

$$S_{\text{node},j} = \frac{1}{|\mathcal{N}_j|} \sum_{i \in \mathcal{N}_j} \left| \frac{\partial f}{\partial p_i} \right| \tag{5}$$

where $\mathcal{N}_j$ is the set of parasitic components connected to node v_j.

To establish a fair threshold for distinguishing between sensitive and insensitive nodes, we compute the mean sensitivity S_{avg} of all nodes by summing up their individual sensitivities and dividing by the total number of nodes.

$$S_{\text{avg}} = \frac{1}{N} \sum_{j=1}^{N} S_{\text{node},j} \tag{6}$$

where N is the total number of nodes. This average sensitivity value S_{avg} serves as the threshold s to classify nodes as sensitive $(S_{\text{node},j} \geq s)$ or insensitive $(S_{\text{node},j} < s)$.

It is worth noting that our method is particularly well-suited for scenarios that involve multiple circuits with the same topology but different parameters, such as the sizing and Monte Carlo analysis. The identification results of essential parasitics can be effectively transferred to similar circuits, further diminishing the perceived overhead of sensitivity analysis. The applicability of our method to these scenarios is demonstrated in the experimental section.

D. Hybrid Strategy to Reduce Simulation Scale

After identifying parasitic-sensitive nodes, we leverage a hybrid strategy to reduce the simulation scale by handling critical and non-critical parasitic sub-networks separately, as illustrated in Figure 4.

Partition the Circuit. The parasitic network extracted by the extraction tool must first be partitioned into N parasitic sub-networks based on pre-layout node. Following the fundamental principles of parasitic formation, the criterion is established as follows: post-layout nodes derived from the same pre-layout node are classified into the same sub-network. In particular, if two post-layout nodes are interconnected through a parasitic resistor, they are regarded as belonging to the same sub-network.

Collapse the parasitic sub-network for nodes that are parasitic-insensitive. To maximize the elimination of non-critical parasitics, a collapsing technique is employed. Specifically, if a node is identified as parasitic-insensitive, all resistors within the parasitic sub-network associated with that node are treated as short-circuit, and all capacitors are treated as open-circuit. The ground capacitors of the sub-network is replaced by an equivalent capacitor, and all internal nodes are merged into a single node. In other words, after transforming the parasitic sub-network into its corresponding matrices, the G matrix is reduced to a first-order zero matrix, while the C matrix is simplified to a first-order matrix, where its element equals sum of the diagonal elements of the original C matrix.

TABLE I: Node reduction and acceleration efficiency comparisons with different methods.

Circuit	#Nodes			Time (ms)			PiSPICE vs Original		PiSPICE vs TICER		Relative Error
	Original	TICER	PiSPICE	Original	TICER	PiSPICE	Reduction Rate	Speedup	Reduction Rate	Speedup	
CKT1 (Ac)	1219	567	16	135.699	40.176	19.686	76.19x	6.89x	35.44x	2.04x	0.05%
CKT1 (Tran)	1219	567	13	189.536	119.603	34.802	**93.77x**	5.45x	**43.62x**	3.44x	0.30%
CKT2 (Ac)	3457	1623	235	616.297	137.021	81.820	14.71x	7.53x	6.91x	1.67x	0.05%
CKT2 (Tran)	3457	1623	568	640.569	261.536	106.922	6.09x	5.99x	2.86x	2.45x	0.16%
CKT3 (Ac)	5048	2447	422	541.906	136.357	53.167	11.96x	10.19x	5.80x	2.56x	0.18%
CKT3 (Tran)	5048	2447	1018	926.216	408.313	140.629	4.96x	6.59x	2.40x	2.90x	0.08%
CKT4 (Ac)	7194	3370	987	3397.520	1159.860	196.770	7.29x	**17.27x**	3.41x	5.89x	0.37%
CKT5 (Ac)	8920	4272	1687	3025.370	733.360	280.880	5.29x	10.77x	2.53x	2.61x	0.34%
CKT6 (Ac)	12255	6329	455	2052.257	447.222	227.815	26.93x	9.01x	13.91x	1.96x	0.25%
CKT7 (Ac)	24306	12743	2171	6430.030	2753.240	888.470	11.20x	7.24x	5.87x	3.10x	0.43%
CKT8 (Tran)	46894	27288	23768	9021930.000	5040080.000	4380000.000	1.97x	2.06x	1.15x	1.15x	**0.78%**
CKT9 (Tran)	68450	37546	29860	10800002.043	6453020.000	679281.022	2.29x	15.90x	1.26x	**9.50x**	0.56%
Average		-			-		21.89x	8.74x	10.43x	3.27x	0.30%

Reduce the size of parasitic sub-networks of parasitic-sensitive nodes with improved PRIMA. Although it is not possible to directly collapse the parasitic sub-networks of parasitic-sensitive nodes, some steps can be taken to reduce its size, i.e., the MOR algorithm can be performed.

In our approach, we apply MOR to non-critical subnetworks, which are ideal candidates for PRIMA due to their smaller size and localized interactions. However, since the parasitic sub-network obtained by partitioning does not necessarily have ground resistor, the resulting G matrix, which is populated by internal interconnect resistor, may not be full-rank or invertible. Therefore, we can not have $R = G^{-1}B$ as the projection matrix like original PRIMA.

To address this issue, we adopt the reordering strategy from sparse implicit projection [37] to enhance PRIMA. By separating the internal and port nodes within each sub-network, the matrix G is reordered into a block matrix form, where A represents the conductance matrix between port nodes, B is the coupling conductance matrix between port and internal nodes, and D is the conductance matrix between internal nodes. This reordering enables us to compute $G^{-1}B$, which leads to the new projection matrix. Through further simplifications, we derive Eq. 7.

$$G = \begin{pmatrix} A & B \\ B^T & D \end{pmatrix}, \quad G^{-1}B = \begin{pmatrix} -A^{-1}B \\ I \end{pmatrix} \tag{7}$$

By using the projection matrix $R = G^{-1}B$ for MOR, the size of critical parasitic sub-network is further reduced.

IV. Experiments

A. Experimental Setup

To comprehensively evaluate the efficiency, accuracy, and applicability of PiSPICE, we use Python to implement the whole framework. The experiment involves a total of nine circuits as test cases, with node counts ranging from 1k to 60k. Specifically, CKT1-CKT6 are operational amplifier circuits of varying scales, CKT7 is a bandgap reference circuit, CKT8 is a 4-bit SAR ADC circuit, and CKT9 is an ultrafast clock fan-out buffer circuit. All experiments are executed on a system equipped with an AMD EPYC 2.3 GHz CPU and 377 GB of memory.

Both transient and AC analysis simulation are conducted. We compare among three different configurations: the original post-layout circuit, the post-layout circuit simplified using TICER, and the post-layout circuit simplified using our method, referred to as "Original", "TICER" and "PiSPICE", respectively. We use Calibre xRC [39] for parasitic extraction and Cadence Spectre [38] as the simulator. TICER is chosen as the baseline for comparison due to its widespread recognition as a reliable MOR method integrated into Calibre xRC, making it a well-respected tool across various design applications.

B. Reduction and Acceleration Efficiency

Table I presents the number of total nodes ($\#Nodes$) in post-layout simulation and the simulation time ($Time(ms)$) for the test circuits across different methods. Additionally, it highlights the improvements in parasitic network reduction and simulation acceleration efficiency achieved by PiSPICE compared to Original and TICER.

From this table, it is clear that PiSPICE achieves a substantial reduction in the number of nodes compared to TICER, due to its aggressive reduction strategy for non-critical parasitic sub-networks. Specifically, for the transient analysis simulation of CKT1, PiSPICE reduces the circuit to 93.77x fewer nodes compared to the original, offering a 43.62x improvement over TICER. This highlights the superior efficiency of PiSPICE in parasitic network simplification.

In addition, it can be also seen that PiSPICE significantly enhances acceleration efficiency by substantially reducing post-layout SPICE simulation time through effective circuit size reduction, thereby expediting the simulation solving process. Specifically, compared to the original post-layout SPICE simulation, i.e., Spectre, PiSPICE achieves a speedup ratio of 2.06x-17.27x. When compared to TICER, PiSPICE also demonstrates a speedup of 1.15x-9.50x.

C. Accuracy Analysis

To verify the accuracy of PiSPICE, we analyze the maximum relative error of the simulation results using our proposed PiSPICE and compare to the original simulation results, denoted as $Relative\ Error$ in Table I. It can be observed that the maximum error introduced by PiSPICE remains below 0.78%, which is deemed acceptable. Figure 5 further presents

the gain and CMRR curves, along with the corresponding absolute error, for CKT1, CKT2, and CKT3 during AC analysis simulation. It can be observed that the gain curves (or CMRR curves) of the original circuit and PiSPICE nearly overlap, with an absolute error of less than 0.6dB, demonstrating that PiSPICE achieves a high level of accuracy.

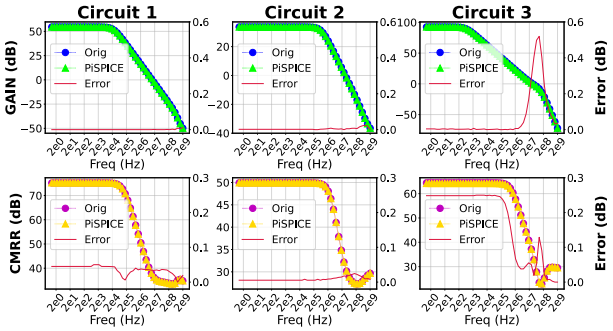

Fig. 5: The gain curves, CMRR curves and absolute error in CKT1, CKT2 and CKT3 under different methods.

TABLE II: The circuit scale and AC simulation accuracy of CKT3 after adjusting s_u.

Circuit	Value of s_u	#Nodes	Time (ms)	Relative Error
CKT3 (Ac)	1.0	10	36.793	3.10%
	0.8	196	48.980	1.10%
	0.5	**422**	**53.167**	**0.18%**
	0.2	2747	336.357	0.06%
	0.0	5048	541.906	0.00%

For PiSPICE, we enable a user-adjustable safety factor s_u to provide more flexibility in adjusting the threshold based on user requirements. When $s_u = 0.5$, it corresponds to the default threshold calculated by our method. When $s_u = 0$, it implies that all nodes are considered parasitic-sensitive, meaning the original post-layout circuit is used without any reduction. On the other hand, when $s_u = 1$, it indicates that all nodes are regarded as non-sensitive to parasitics and can be eliminated, resulting in a circuits identical to the pre-layout circuit. Table II presents the performance and error results after dynamically adjusting s_u. Users can balance the relaxation of sensitivity judgment between desired accuracy and speed.

D. Applicability Analysis

As mentioned in section III. C, our method achieves greater performance improvements in scenarios where the topology is the same but the parameters differ, such as in sizing. We only need to perform sensitivity analysis once and then transfer this information to similar circuits, significantly reducing the cost of sensitivity analysis. Here, we use CKT3 as an example. CKT3_1 in Table III represents the original circuit, while CKT3_2 to CKT3_4 are three similar circuits where transistor parameters are adjusted during the sizing process. We perform sensitivity analysis on CKT3_1 to identify the critical nodes in the circuit, and then use this information to directly reduce the RC networks of the other three circuits.

Table III shows the simulation results. As shown in the table, all the three netlists, CKT3_2 to CKT3_4, achieve significant performance improvements, with an average simulation speedup of 8.34x, while maintaining errors within 0.5%. Similar conclusions can be drawn from the reduction in circuit size (i.e., number of total nodes). Compared to directly obtain the RC networks from Calibre xRC, our method results in an average reduction of 13.1x in the size of the post-layout netlist.

TABLE III: Performance of PiSPICE in post-layout simulation with the same topology but different transistor parameters.

Circuit	#Nodes		Time (ms)			Speedup	Relative Error
	Original	PiSPICE	Original	PiSPICE			
CKT3_1 (Ac)	5048	422	541.906	53.167	**Sensitivity**	10.19x	0.18%
CKT3_2 (Ac)	3955	157	287.349	37.225	**Analysis**	7.72x	0.26%
CKT3_3 (Ac)	4699	976	323.481	50.390	**Time:**	6.42x	0.20%
CKT3_4 (Ac)	21262	2053	1805.040	199.978	**221.526**	9.03x	0.50%
Average	-		-			8.34x	0.29%
Time Sum	-		**2957.776**	**562.286**		5.26x	-

E. Overhead of Sensitive Analysis

Although our method significantly reduces the post-simulation circuit size and improves simulation efficiency, sensitivity analysis does come with a certain computational cost. As shown in Table III, the cost of sensitivity analysis in our method ranges from approximately 0.12x to 0.77x of a single simulation run (compared to the original simulation time). Fortunately, in scenarios where sensitivity results information is transferred across similar circuits, the total simulation time for four circuits (including the preprocessing cost of sensitivity analysis and the simulations themselves) still achieves a performance improvement of 5.26x. If more netlists that share the same topology need to be simulated during the sizing optimization process, the cost of sensitivity analysis becomes negligible.

V. Conclusion

In this paper, we propose PiSPICE, a novel framework designed to accelerate post-layout SPICE simulation by identifying and retaining only the essential parasitics. First, we leverage the topological correlation between pre- and post-layout circuits for parasitic modeling. Next, critical parasitics are identified through sensitivity analysis of the parasitic model. Finally, we collapse the critical parasitic sub-networks of parasitic-insensitive nodes, while the parasitic sub-networks of parasitic-sensitive nodes are reduced using a modified PRIMA approach. Experiment results demonstrate that PiSPICE achieves superior acceleration while maintaining comparable accuracy to the commercial simulator Spectre with TICER.

Acknowledgment

This work was supported by NSFC (Grant No.62234010, 92473107, 62204265, U23A20301, 62374031), Beijing National Research Center for Information Science and Technology (BNRist), Semiconductor Technology Innovation Center (Beijing) (Grant No.QYJS-2021-0900-B) and NSF of Jiangsu Province (BK20240173). Dan Niu and Zuochang Ye are corresponding authors.

REFERENCES

[1] T. Nakura, "SPICE Simulation," *Essential Knowledge for Transistor Level LSI Circuit Design*, pp. 19-47, 2016.

[2] D. Niu, Y. Dong, Z. Jin, C. Zhang, Q. Li and C. Sun, "OSSP-PTA: An Online Stochastic Stepping Policy for PTA on Reinforcement Learning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 42, no. 11, pp. 4310-4323, 2023.

[3] T. Wang, W. Li, H. Pei, Y. Sun, Z. Jin and W. Liu, "Accelerating Sparse LU Factorization with Density-Aware Adaptive Matrix Multiplication for Circuit Simulation," *ACM/IEEE Design Automation Conference (DAC)*, pp. 1-6, 2023.

[4] Z. Jin, W. Li, Y. Bai, T. Wang, Y. Lu and W. Liu, "Machine Learning and GPU Accelerated Sparse Linear Solvers for Transistor-level Circuit Simulation: A Perspective Survey (Invited Paper)," *Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 96–101, 2024.

[5] Z. Jin, T. Feng, X. Wu, D. Niu, Z. Zhou and C. Zhuo, "MSH: A Multi-Stage HiZ-Aware Homotopy Framework for Nonlinear DC Analysis," *Design, Automation and Test in Europe Conference (DATE)*, pp. 1-6, 2024.

[6] Z. Jin, H. Pei, Y. Dong, X. Jin, X. Wu, W.W. Xing and D. Niu, "Accelerating Nonlinear DC Circuit Simulation with Reinforcement Learning," *ACM/IEEE Design Automation Conference (DAC)*, pp. 619–624, 2022.

[7] W.W. Xing, X. Jin, T. Feng, D. Niu, W. Zhao and Z. Jin, "BoA-PTA: A Bayesian Optimization Accelerated PTA Solver for SPICE Simulation," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 28, no. 2, pp. 1-26, 2022.

[8] Y. Chen, H. Pei, X. Dong, Z. Jin and C. Zhuo, "Application of Deep Learning in Back-End Simulation: Challenges and Opportunities," *Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 641-646, 2022.

[9] G. Feng, H. Wang, Z. Guo, M. Li, T. Zhao, Z. Jin, W. Jia, G. Tan and N. Sun, "Accelerating Large-Scale Sparse LU Factorization for RF Circuit Simulation," *International European Conference on Parallel and Distributed Computing (Euro-Par)*, pp. 182-195, 2024.

[10] Y. Dong, D. Niu, Z. Jin, C. Zhang, C. Sun and Z. Zhou, "ISPT-Net: A Noval Transient Backward-Stepping Reduction Policy by Irregular Sequential Prediction Transformer," *Design, Automation, and Test in Europe Conference (DATE)*, pp. 1-6, 2024.

[11] Y. Bai, X. Yang, Y. Lu, D. Niu, C. Zhuo, Z. Jin and W. Liu, "Efficient Spectral-Aware Power Supply Noise Analysis for Low-Power Design Verification," *Design, Automation, and Test in Europe Conference (DATE)*, pp. 1-6, 2024.

[12] X. Fu, B. Zhang, T. Wang, W. Li, Y. Lu, E. Yi, J. Zhao, X. Geng, F. Li, J. Zhang, Z. Jin and W. Liu, "PanguLU: A Scalable Regular Two-Dimensional Block-Cyclic Sparse Direct Solver on Distributed Heterogeneous Systems," *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pp. 1-15, 2023.

[13] J. Zhao, Y. Wen, Y. Luo, Z. Jin, W. Liu and Z. Zhou, "SFLU: Synchronization-Free Sparse LU Factorization for Fast Circuit Simulation on GPUs," *ACM/IEEE Design Automation Conference (DAC)*, pp. 37-42, 2021.

[14] A. Zhao, T. Gu, Z. Bi, F. Yang, C. Yan, X. Zeng, Z. Lin, W. Hu and D. Zhou, "D3PBO: Dynamic Domain Decomposition-Based Parallel Bayesian Optimization for Large-Scale Analog Circuit Sizing," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 29, no. 44, pp. 1-25, 2024.

[15] W. Yu, T. Zhang, X. Yuan and H. Qian, "Fast 3-D Thermal Simulation for Integrated Circuits With Domain Decomposition Method," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 32, no. 12, pp. 2014-2018, 2013.

[16] F. Bizzarri, A. Brambilla and G. Storti Gajani, "FastSpice Circuit Partitioning to Compute DC Operating Points Preserving Spice-Like Simulators Accuracy," *Simulation Modelling Practice and Theory*, vol. 81, pp.51-63, 2018.

[17] Z. Jin, T. Feng, Y. Duan, X. Wu, M. Cheng, Z. Zhou and W. Liu, "PALBBD: A Parallel ArcLength Method Using Bordered Block Diagonal Form for DC Analysis," *ACM Great Lakes Symposium on VLSI (GLSVLSI)*, pp. 327-332, 2021.

[18] Y. Jiang, J. Song, X. Yang, X. Dong, S. Sun, Y. Lin, Z. Jin, X. Yin and C. Zhuo, "A Parallel Simulation Framework Incorporating Machine Learning-Based Hotspot Detection for Accelerated Power Grid Analysis," *ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD)*, pp. 1-7, 2024.

[19] Y. Su, F. Yang and X. Zeng, "AMOR: An Efficient Aggregating Based Model Order Reduction Method for Many-Terminal Interconnect Circuits," *Design Automation Conference (DAC)*, pp. 295–300, 2012.

[20] J. Rommes and W. Schilders, "SparseRC: Sparsity Preserving Model Reduction for RC Circuits with Many Terminals," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 30, pp. 1828-1841, 2011.

[21] D. Oyaro and P. Triverio, "TurboMOR-RC: An Efficient Model Order Reduction Technique for RC Networks With Many Ports," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 35, pp. 1695-1706, 2016.

[22] P. Liu, S.X.-D. Tan, B. McGaughy, L. Wu and L. He, "TermMerg: An Efficient Terminal-Reduction Method for Interconnect Circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 26, no. 8, pp. 1382-1392, 2007.

[23] Z. Zhang, X. Hu, C. -K. Cheng and N. Wong, "A Block-Diagonal Structured Model Reduction Scheme for Power Grid Networks," *Design, Automation and Test in Europe Conference (DATE)*, pp. 1-6, 2011.

[24] P. Chen, D. Niu, Z. Jin, C. Sun, Q. Li and H. Yan, "TSA-TICER: A Two-Stage TICER Acceleration Framework for Model Order Reduction, "*Design, Automation, and Test in Europe Conference (DATE)*, pp. 1-6, 2024.

[25] Z. Zhang, D. Niu, Z. Jin, P. Chen, Z. Zhou and C. Sun, "P-TICER: An Effective Parallel TICER Acceleration Method for Model Order Reduction," *International Symposium of Electronics Design Automation (ISEDA)*, pp. 125-130, 2024.

[26] L. Hao and G. Shi, "Realizable Reduction of Multi-Port RCL Networks by Block Elimination," *IEEE Transactions on Circuits and Systems I (TCAS-I)*, vol. 70, no. 1, pp. 399-412, 2023.

[27] S. Director and R. Rohrer, "Automated Network Design-The Frequency Domain Case," *IEEE Trans. Circuit Theory*, vol. 16, no. 3, pp. 330–337, 1969.

[28] O. Aydogmus and A.TOR, "A Modified Multiple Shooting Algorithm for Parameter Estimation in ODEs Using Adjoint Sensitivity Analysis," *Applied Mathematics and Computation*, vol. 390, pp. 125644, 2021.

[29] A. R. Conn, P. K. Coulman, R. A. Haring, G. L. Morrill, C. Visweswariah and C. Wu, "JiffyTune: Circuit Optimization Using Time-Domain Sensitivities," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 17, pp. 1292-1309, 1998.

[30] Z. Gao and R. Rohrer, "Efficient Non-Monte-Carlo Yield Estimation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 41, no. 5, pp. 1222–1235, 2022.

[31] W. Hu, Z. Wang, S. Yin, Z. Ye and Y. Wang, "Sensitivity Importance Sampling Yield Analysis and Optimization for High Sigma Failure Rate Estimation," *ACM/IEEE Design Automation Conference (DAC)*, pp. 895-900, 2021.

[32] W. Hu, Z. Ye and Y. Wang, "Adjoint Transient Sensitivity Analysis for Objective Functions Associated to Many Time Points," *ACM/IEEE Design Automation Conference (DAC)*, pp. 1-6, 2020.

[33] J. Li, D. Ahsanullah, Z. Gao and R. Rohrer, "Circuit Theory of Time Domain Adjoint Sensitivity," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 42, pp. 2303-2316, 2023.

[34] C. Li, B. Zhang, Y. Duan, Y. Li, Z. Ye, W. Liu, D. Tao and Z. Jin, "MASC: A Memory-Efficient Adjoint Sensitivity Analysis through Compression Using Novel Spatiotemporal Prediction," *ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6. 2024.

[35] A. Odabasioglu, M. Celik and L. Pileggi, "PRIMA: Passive Reduced-Order Interconnect Macromodeling Algorithm," *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp. 58–65, 1997.

[36] B. N. Sheehan, "TICER: Realizable Reduction of Extracted RC Circuits," *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp. 200-203, 1999.

[37] Z. Ye, D. Vasilyev, Z. Zhu and J. Phillips, "Sparse Implicit Projection (SIP) for Reduction of General Many-Terminal Networks," *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp. 736–743, 2008.

[38] Cadence, "Cadence Spectre Simulation Platform, " Cadence. [Online]. Available: https://www.cadence.com.

[39] Siemens, "Calibre xRC," Siemens. [Online]. Available: https://eda.sw.siemens.com.